

Contents

Governing Agentic Software Development with Executable Evidence	3
Mechanical Gates, Human Review, and Skill Capitalization	3
以可執行證據治理 Agentic 軟體開發: 機械閘、人類審查與技能資產化	3
(內部代號 / internal codename) ALE —TigerAI (與 arXiv:2509.06216 之 “Agentic Loop Engineering” 無關)	3
版本沿革 (Version Lineage)	3
相對 v2.3 的修訂摘要 (Changelog v2.3 → v2.4)	4
摘要 (Abstract · 中文)	5
Abstract (English)	5
1. 引言 (Introduction)	6
1.1 動機	6
1.2 一個被忽略的新風險	6
1.3 貢獻 (Contributions)	6
1.4 研究問題 (Research Questions)	6
1.5 新穎性聲明 (Novelty Statement)	6
2. 背景與相關研究 (Background & Related Work)	7
2.1 DevOps / DevSecOps 與軟體交付度量	7
2.2 Agentic / 多代理人 LLM 系統	7
2.3 軟體測試的事實基準	7
2.4 集成學習與估計器相關性	7
2.5 威脅建模與技能互通協定	7
2.6 LLM 評審的自我偏好與 Agentic SWE 評測	7
2.7 Loop Engineering、Agentic SWE、測試 Oracle 與多代理人共謀 (與本文之區別)	8
3. ALE 框架定義 (Framework Definition)	9
3.1 核心定義	9
3.2 定位	9
3.3 三層抽象	9
3.4 用語上的審慎	9
3.5 適用邊界 (Scope Boundary)	9
4. ALE 流程閉環 (The ALE Loop)	10
4.1 完整流程	10
4.2 各階段的交付物與門檻 (節錄)	10
4.3 核心鐵則	11
5. 五大核心模組 (Five Core Modules)	11
5.1 模組一: ALE Process (主流程管線)	11
5.2 模組二: ALE Repository System (四大工程倉庫)	11
5.3 模組三: ALE Agent Roles (多代理人生產線)	11
5.4 模組四: ALE Skill Manifest (標準能力單元格式)	12
5.5 模組五: ALE Skill Lifecycle (技能生命週期治理)	12
5.6 Agent 權限模型 (Agent Permission Model)	12
6. 技能資產化作為核心命題 (Skill Capitalization)	13

7. 反自我欺騙的可驗證治理 (Anti-Self-Deception Governance)	13
7.1 失效模式: 共享脈絡假通過 (Shared-Context False Pass; 早期稱 Test Collusion)	13
7.2 為何多模型交叉檢查可能不足:variance、bias 與相關性 [本節主張為 [研究假說]]	14
7.3 機械閘: 較不依賴模型意見的可執行證據	15
7.4 三層驗證架構	15
7.5 核心守則	15
7.6 自我修正的安全閘	16
7.7 需求不變量: 封堵 Garbage-In 的源頭防禦 (Spec Invariants / Metamorphic Relations · v2.3 新增)	16
8. 安全、合規與數據主權 (Security, Compliance & Data Sovereignty)	17
8.5 技能供應鏈安全 (Skill Supply Chain Security)	17
8.6 ALE 產線本身的威脅模型 (Threat Model for the ALE Pipeline)	17
8.7 AI 特有威脅: Prompt Injection 與 Agent Hijacking 的產線級防禦 (v2.3 新增)	18
9. 工程落地規格 (Engineering Realization)	18
9.1 n8n 狀態機: 動態路由與斷點續傳	18
9.2 代理人通訊協定: JSON-RPC	19
9.2.1 Skill Curator 去退化演算法 (Conflict Resolution Tactics · v2.3 新增)	19
9.3 狀態與證據的分離	19
9.4 參考技術棧	20
9.5 元迴圈: 監控產線本身 (Kaizen)	20
9.6 機械閘的成本工程 (Cost Engineering of the Mechanical Gate)	20
9.7 Claim-Evidence Matrix (主張 證據對照)	20
10. 驗證計畫 (Evaluation Plan)	21
11. ALE 成熟度模型 (ALE Maturity Model)	22
12. 討論 (Discussion)	23
12.1 與一般 AI Coding 的差異	23
12.2 限制與威脅效度 (Limitations & Threats to Validity)	23
12.3 倫理與責任	24
13. 未來工作 (Future Work)	24
13.6 公開出版前待辦 (Pre-publication Checklist)	24
14. 結論 (Conclusion)	24
附錄 A : ALE Skill Manifest 範本 (v2.3)	25
附錄 B : 建議倉庫結構	26
Generative AI Use Disclosure (生成式 AI 使用揭露)	26
附錄 C : Artifact 與資料可得性 (Artifact & Data Availability)	27
參考文獻 (References)	27

Governing Agentic Software Development with Executable Evidence

Mechanical Gates, Human Review, and Skill Capitalization

以可執行證據治理 Agentic 軟體開發：機械閘、人類審查與技能資產化

(內部代號 / internal codename) ALE —TigerAI (與 arXiv:2509.06216 之 “Agentic Loop Engineering” 無關)

作者 / Author : Yeh-Hsing Lu (Morris; 虎智科技 Tiger AI Tech Co., Ltd.) 文件類型 / Type : White Paper · Research Framework (技術白皮書 / 框架研究) 版本 / Version : v3.0 (重大更名 + 依 Codex DOI 前審做三項事實修正; 見下方 Changelog) 日期 / Date : 2026-06-21 狀態 / Status : Working Draft / Conceptual Framework and Research Agenda —含初步田野觀察; 受控實驗 (EXP-001/002) 尚未完成。

引用建議格式 : Lu, Y.-H. (2026). *Governing Agentic Software Development with Executable Evidence: Mechanical Gates, Human Review, and Skill Capitalization* (v3.0, Working Paper). Zenodo (DOI 待鑄). 標題已由作者拍板 (2026-06-21, 採命名方案 B); 內部代號 ALE 保留。

重大更名 (v3.0): 原標題「System Development Agentic Loop Engineering (ALE)」棄用, 因 arXiv:2509.06216(Agentic Software Engineering Roadmap)§5.2 已正式定義 “Agentic Loop Engineering (ALE)” (同領域、同全名、同縮寫)。為免學術引用歧義, 本文改用上方新標題 (作者已拍板, 命名方案 B; 以 *Mechanical Gates / Human Review / Skill Capitalization* 為關鍵詞利於檢索); 「ALE」僅作 TigerAI 內部代號, 與 arXiv:2509.06216 的 ALE 無關 (本文不宣稱 ALE 為學術命名原創)。

主張類型標註慣例: 核心命題標註證據等級:[設計原則] / [工程規格] / [研究假說] / [初步證據] / [已驗證]。對應全表見 §9.7 Claim–Evidence Matrix。截至本版, 尚無 [已驗證] 等級之主張; 核心命題多為 [研究假說] 或 [初步證據], 待 EXP-001/002(見 ../04_Evidence/) 產生直接資料。

Generative AI 使用揭露: 本文以多 AI 引擎協作生產 (Claude 操作化/查核、Codex 對抗式審查、Gemini 形式化/潤稿), 研究問題、田野證據、引文查核與最終責任由人類作者承擔, AI 不列為作者。完整方法論與揭露見 ../08_Methodology/。

版本沿革 (Version Lineage)

ALE v1.1 : 讓 Agent 接手 SDLC (產線骨架)

ALE v1.2 : 讓 Agentic SDLC 可驗證 (機械閘 / 反共謀 / 三層驗證)

ALE v2.0 : 讓 Agentic SDLC 可治理、可稽核、可沉澱

ALE v2.1 : 讓主張可被研究驗證、且可作為企業治理框架落地

(RQ / Evaluation Plan / Maturity Model / 權限模型 / 供應鏈安全 / 產線威脅模型)

ALE v2.3 : 強化統計論證、需求源頭防禦與 AI 特有威脅模型 (審稿人攻防版)

ALE v2.4 : 依 Codex 審閱校準主張強度; 標註證據等級、加 Claim–Evidence Matrix、納入 Related Work、凍結 EXP-001

ALE v2.5 : 嫁接 Gemini 三項強化 (§7.7 Python 不變量引擎、§11 數字 KPI、正式文獻)

v3.0 : ** 重大更名 ** (讓出與 arXiv:2509.06216 撞名的 ALE) + 三項事實修正 + GenAI 揭露

v2.5 → v3.0 改了什麼 (依 Codex DOI 前審, 3 項皆經作者讀原文查證): 1. **更名:** 棄用「Agentic Loop Engineering (ALE)」(arXiv:2509.06216 §5.2 已定義同名); 改「From Model Consensus to Executable Evidence」。2. **修正 arXiv:2512.03097 誤讀:** 原文是「接外部指南的 verifier 把攻擊成功率 100%→0%」(verifier 有效), 非「verifier 擋不住」; 改寫為「模型共識 安全, 外部證據驗證才有效」——此反而更支持本文機械閘論點 (§2.7、§7.2)。3. **補直接 prior art:** arXiv:2603.15611(Code-A1)

已用 *self-collusion* 描述 code/test 模型同類問題;test collusion 改稱 *shared-context false pass*, 並降級新穎性宣稱 (§1.5、§2.7、§7.1)。4. 摘要/貢獻改假說語氣; 加 GenAI 使用揭露 (§0.1);Validated→internally assessed;KPI→organization-defined。v2.4/v2.5 之治理深度 (§5.6/§7.4/§8.5-8.7/§9.x/§12.2) 全數保留。

相對 v2.3 的修訂摘要 (Changelog v2.3 → v2.4)

本版**不新增概念**，專做 Codex 審閱指出的「主張強度 證據強度 引文」對齊。核心改動：

項目	類型	位置	來源
移除「無懈可擊」等不可證偽措辭	校準	版本沿革、§14	Codex P0-1
§5.3「自我審查近乎無效」降級為「角色/脈絡/證據來源獨立性」	校準	§5.3	Codex P0-2
test collusion 重定義為 correlated false-pass failure mode; 新穎性降為「候選貢獻」	校準	§1.5、§7.1	Codex P0-3
§7.2 統計表述標為 研究假說 ，定義可觀測的錯誤相關性，指向 EXP-001 量測	校準	§7.2	Codex P1-6
機械閘 ground truth; 改為「可執行、可重現、較不依賴模型意見的證據」	校準	摘要、§7.3、§7.4	Codex P1-7
§9.2.1 =0.85 標為初始政策;AST 改為 artifact-specific similarity	校準	§9.2.1	Codex P1-8
成熟度模型補各級 entry criteria / gates / evidence / KPI	強化	§11	Codex P1-9
新增 §2.7 Related Work (loop engineering / collusion / oracle 文獻)	新增	§2.7	PriorArt 稿
新增 §9.7 Claim-Evidence Matrix	新增	§9.7	Codex P2-10
引用補強 + 新增 §13.6 公開前待辦 (引文核驗、EXP-001)	新增	References、§13.6	Codex P0-4

誠實聲明：本 v2.4 已完成「降低過度宣稱」之校準，但**尚未**滿足 Codex 所列之公開出版門檻——EXP-001 尚未執行、部分引文待全文核驗。故狀態仍為 Working Draft。v2.1/v2.3 全數結論保留（僅措辭校準，非推翻）。

摘要 (Abstract · 中文)

大型語言模型 (LLM) 已能在單次互動中產出可執行的程式碼，但企業級軟體交付要求的並非「一次性可運行」，而是**可治理、可稽核、可回滾、可重複**的工程能力。本文提出一套**證據治理的 Agentic 軟體開發生命週期**框架 (TigerAI 內部代號 ALE)，將完整 SDLC 映射到多代理人 (multi-agent) 自主協作流水線。核心主張有三：(1) **全生命週期閉環**；(2) **知識資產化**——每完成一專案即萃取可重用、可測試、可治理的「技能單元 (Skill)」，以有限狀態機治理其生命週期；(3) **反自我欺騙的可驗證治理**——我們關注一種失效模式：當產生程式與驗證程式的代理人**共享同一實作脈絡/偏誤源**時，可能出現**相關性假通過 (shared-context false pass; 我們早期稱 test collusion)**。我們提出研究假說：多模型交叉檢查可降低變異 (variance)，但在共享偏誤下未必能消除系統性偏誤 (bias)——此假說待 EXP-001 直接量測，本文不作為已成立事實。我們因此把設計重心放在以**可執行證據取代模型共識**作為 release 決策依據：以**機械閘 (mechanical gate)**(突變測試、覆蓋率、性質測試、需求不變量、真實執行) 提供**可執行、可重現、較不依賴模型意見**的證據 (強證據，但不單獨保證需求完整正確)；**模型評議 (model panel)** 僅用於主觀判斷關卡；並令**人類審查證據而非共識**。值得注意，近期實證 [arXiv:2512.03097] 顯示：多代理人**共識**本身不等於安全，但**接上外部可信指南的 verifier** 能有效防禦——這與本文「驗證的價值來自外部證據，而非它也是另一個模型」一致。本文同時給出以 n8n 狀態機與 JSON-RPC 代理人協定為基礎的工程實作規格，並討論其在受監管 (如醫療地端) 情境對數據主權、Prompt Injection 防禦與合規的意義。

關鍵詞：Agentic AI、軟體開發生命週期、多代理人系統、LLM 程式生成、可驗證性、突變測試、需求不變量、技能資產化、技能供應鏈安全、Prompt Injection、DevSecOps、數據主權

Abstract (English)

Large language models (LLMs) can produce runnable code within a single interaction, yet enterprise software delivery requires not “runs once” but **governable, auditable, reversible, and repeatable** engineering capability. This paper proposes an **evidence-governed lifecycle for agentic software development** (TigerAI internal codename: ALE) that maps the full SDLC onto an autonomous, multi-agent pipeline. It rests on three claims: (1) a **closed full-lifecycle loop**; (2) **knowledge capitalization**, whereby each completed project yields reusable, testable, governable *Skill* units governed by a finite-state machine; and (3) **anti-self-deception governance**. We study a failure mode in which code-generating and verification agents that share the same implementation context/bias source may produce a **shared-context false pass** (which we earlier called *test collusion*). We **hypothesize** that multi-model cross-checking reduces *variance* but, under shared bias, may not remove systematic *bias*; this is stated as a research hypothesis pending direct measurement (EXP-001), not as established fact. The framework therefore **privileges executable evidence over model consensus** at release decisions: a **mechanical gate** (mutation testing, coverage, property-based testing, spec invariants, real execution) supplies executable, reproducible evidence less dependent on model opinion (strong, but not by itself a correctness oracle); a **model panel** is used only at judgment-bearing checkpoints; and **humans review evidence rather than consensus**. Notably, recent empirical work [arXiv:2512.03097] shows that multi-agent *consensus* alone is not safety, whereas a **verifier anchored to trusted external guidelines** effectively defends—consistent with our position that a verifier’s value comes from external evidence, not from being yet another model. We give an engineering specification grounded in an n8n state machine and a JSON-RPC inter-agent protocol, and discuss implications for data sovereignty, prompt-injection defense, and compliance in regulated, on-premises settings such as healthcare.

Keywords: Agentic AI, software development life cycle, multi-agent systems, LLM code generation, verifiability, mutation testing, spec invariants, skill capitalization, skill supply-chain security, prompt injection, DevSecOps, data sovereignty

1. 引言 (Introduction)

1.1 動機

目前主流的「AI 寫程式」實務，可化約為以下短鏈：

Prompt → Code → Debug

此模式在原型 (prototype) 與教學示範上有效，但放到企業級交付時暴露四個結構性缺陷：**產出不可預測、缺乏架構審查、技術債野蠻增生、知識無法沉澱**，且**全程缺乏可稽核的證據軌跡 (evidence trail)**。我們長期觀察到一個與工具無關的洞察：企業導入 AI 的真正瓶頸往往不在工具是否強大，而在於人的工程能力與治理機制是否到位。因此，問題不是「讓 AI 把程式寫得更好」，而是「讓 AI 能夠在可治理的框架下，接手整條軟體生產線」。

1.2 一個被忽略的新風險

當我們把更多生命週期階段交給 AI Agent——包括**驗證階段**——時，出現一個新的、且相當隱性的風險：**驗證 AI 的也是 AI**。若由一個 Agent 產生程式、另一個 Agent 為該程式撰寫測試，兩者會收斂到「測試全數通過、但實質上未驗證任何規格」的均衡。本文將此命名為**測試共謀 (test collusion)**，並主張它不是靠「多找幾個模型互相檢查」就能解決——這是本文的核心論點之一（第 7 節）。

1.3 貢獻 (Contributions)

1. **框架形式化**：提出 ALE，將 SDLC 形式化為帶有交付物、驗收門檻與回饋閉環的多代理流水線 (§3-4)。
2. **技能資產化模型**：將「專案經驗萃取為可重用技能」形式化為具標準化 Manifest 與有限狀態機治理的生命週期，並提出去退化（去重/合併/契約回歸）演算法 (§5-6、§9.2.1)。
3. **反自我欺騙的可驗證性理論與機制**：界定 test collusion；以 variance/bias、correlated estimators 與條件協方差論證多模型交叉檢查的能力邊界（含形式化）；提出「機械閘 / 模型評議 / 人類審證據」三層驗證，並以**需求不變量**封堵需求源頭盲區 (§7)。
4. **企業治理與 AI 特有安全規格**：Agent 權限模型、技能供應鏈安全、產線威脅模型、Prompt Injection / Agent Hijacking 防禦，以及 n8n 狀態機與 JSON-RPC 實作藍圖 (§5、§8、§9)。
5. **可被驗證的研究設計**：明確 Research Questions 與 Evaluation Plan，使主張可被後續受控研究檢驗 (§1.4、§10)。

1.4 研究問題 (Research Questions)

- **RQ1 (共謀存在性)**：test collusion 是否會導致「測試通過但規格未被真正驗證」？其發生率與嚴重度如何？
- **RQ2 (機械閘有效性)**：mechanical gates(突變測試、覆蓋率、性質測試、需求不變量) 是否比 multi-model review 更有效降低 escaped defects？
- **RQ3 (技能資產化效益)**：Skill Repository 是否能降低後續專案的重工率、交付時間與錯誤率？
- **RQ4 (治理可稽核性)**：Evidence / Policy Repository 是否能提升 AI 生成軟體的可稽核性與可治理性？
- **RQ5 (人機分界)**：在受監管產業中，哪些階段必須保留 human gate，哪些可安全自動化？

1.5 新穎性聲明 (Novelty Statement)

本文**明確不主張發明** agent loop、loop engineering、SDLC、guardrail、mutation testing，或「code/test 共謀」這個現象本身——後者已見於 **Code-A1 [arXiv:2603.15611]** 的 *self-collusion*(見 §2.7)。本文的**候選貢獻 (candidate contributions)** 是**整合而非首創**：(1) **Agentic SDLC**——SDLC 階段映射為多 Agent 可執行、可審查、可回滾的生產線；(2) **Skill Capitalization**——專案經驗形式化為可測試、可治理的 Skill Manifest 與 Lifecycle；(3) **Evidence-Governed Verification**——把「共享脈絡假通過 (shared-context false pass)」放進完整 Agentic SDLC 治理，並以 **mechanical evidence**(而非更多模型共識) 作 admission/release gate，輔以 model disagreement 與 human evidence review 的權責分層、evidence repository、rollback 與 skill lifecycle 整合。

用詞審慎：稱「候選貢獻」而非「貢獻」，因須待 EXP-001/002 與系統性文獻回顧 (§2.7、§13.6) 確認後方升級。對外一句話：*From model consensus to executable evidence —an evidence-governed lifecycle for agentic software development.* 與 Code-A1 的差異：Code-A1 解的是「兩個模型 (code/test) 如何不互相放水」；本文解的是「整條 SDLC 如何以可執行證據治理、可稽核、可回滾、可沉澱」。

針對「這不就是 DevOps + Agent?」的預期質疑，本文立場：ALE 的核心是 **Agentic SDLC × Skill Capitalization × Anti-Self-Deception Governance** 的交集，尤其第三點是既有研究較少整合處理之處。

2. 背景與相關研究 (Background & Related Work)

2.1 DevOps / DevSecOps 與軟體交付度量

ALE 在精神上延續 DevOps/DevSecOps 整合開發、安全與維運的理念，承襲「安全左移」與「以度量驅動改善」[Kim et al. 2016; Forsgren et al. 2018]。差異在於：傳統 DevOps 執行者是人類團隊，而 ALE 的執行者是受治理的 AI Agent 群，因此必須額外處理「自主代理人的可信驗證」。

2.2 Agentic / 多代理人 LLM 系統

近年代理化方法——ReAct [Yao et al. 2023]、多代理人辯論 [Du et al. 2023]、自我一致性取樣 [Wang et al. 2022]——展示了多步驟協作潛力。ALE 借鑑這些模式組織角色分工與自我修正，但指出 (§7)：它們多半改善變異而非系統性偏誤，在驗證關卡上不能作為事實基準。

2.3 軟體測試的事實基準

突變測試 [DeMillo, Lipton & Sayward 1978]、性質導向測試 [Claessen & Hughes 2000] 與蛻變測試 (**metamorphic testing**) [Chen et al. 1998] 提供了不依賴主觀判斷的測試充分性與正確性度量。ALE 將此類機械度量提升為驗證關的「定生死」依據 (§7.3、§7.7)。

2.4 集成學習與估計器相關性

集成方法的有效性取決於成員的多樣性與相對獨立性 [Dietterich 2000]。本文據此論證：當代 LLM 訓練語料與歸納偏好高度重疊，將其視為「相關性高的估計器」更為貼切，以模型集成取代機械事實基準，風險在於得到「更有自信的一致錯誤」。

2.5 威脅建模與技能互通協定

採 STRIDE 類威脅建模 [Shostack 2014] 作為設計階段安全左移工具，並以 Model Context Protocol(MCP)[Anthropic 2024] 作為技能封裝與跨代理人調用的候選介面之一。

2.6 LLM 評審的自我偏好與 Agentic SWE 評測

兩條與 §7 直接相關的近期線索：

- **LLM-as-judge 的自我偏好偏誤 (self-preference bias)**: LLM 評審存在位置偏誤、冗長偏誤與偏好自身風格輸出之現象。這直接佐證 §7.2: 當評審與被評審共享相近訓練分佈時，模型評議是**相關性高的估計器**，不能作為「定生死」基準。
- **Agentic SWE evals**: 以真實 repository 任務評估 Agent 端到端解題能力的基準 (issue-to-patch 類)，聚焦**生成能力**。ALE 與之互補——它們衡量「能不能解出來」, ALE 處理「解出來後如何治理與可信驗證」。

研究定位：相較於既有研究多聚焦生成能力, ALE 聚焦其互補面——**生成之後的治理與可驗證性**。本文刻意把 LLM-as-judge 自我偏好文獻當作支援證據：它說明了「為什麼模型評議不能定生死」。

2.7 Loop Engineering、Agentic SWE、測試 Oracle 與多代理人共謀（與本文之區別）

近期實務社群將「**loop engineering**」界定為設計與維護 agent 的 plan-act-observe 迴圈，並以 guardrails 控制其行為 [Data Science Dojo 2026 等]。學術側，[**Agentic SWE Roadmap, arXiv:2509.06216**] §5.2 已正式定義 “**Agentic Loop Engineering (ALE)**” 為其工程活動之一。本文因此棄用該名稱（見標題），改以「模型共識 → 可執行證據」立論；本文之候選貢獻不在迴圈本身，而在迴圈產出之治理與可驗證性。

本文的 **shared-context false pass** (早期稱 **test collusion**) 與下列既有議題相鄰，屬其特化/整合而非全新發現：

- **直接 prior art:code/test self-collusion** [Code-A1, arXiv:2603.15611, 2026-03]: 已指出 white-box 下單一模型自我對局會產生 *self-collusion* (產 trivial tests 換 reward)，解法是將 **Code LLM** 與 **Test LLM** 目標對立分離。這與本文 PG V&V 的問題與「規格隔離」對策高度重疊；本文不再宣稱此現象之命名或特化為原創，改定位為「把它整合進完整 Agentic SDLC 治理」。
- **測試 Oracle 問題** [Barr et al. 2015]: 本文需求不變量/蛻變關係是 partial oracle 之運用，非新創。
- **共因失效 / 測試套件過擬合**：相關工程概念。
- **LLM-as-judge 偏誤** [Zheng et al. 2023]: 模型評審有系統性偏誤，但原文亦指出偏誤可被緩解，故不能據以推論「所有 model panel 皆無效」。
- **多代理人共謀**:[arXiv:2512.03097] 在醫療情境顯示——多代理人共識本身不安全（攻擊成功率達 100%），但接上外部可信指南的 **verifier** 將其壓至 0%。正確解讀：**verifier** 之所以有效，是因為它接了外部證據，而非因為它也是一個模型；這正面支持本文「以外部/可執行證據取代模型共識」之主軸。（另:[arXiv:2603.20281] 主題偏定價演算法共謀，相關性較弱；[arXiv:2510.04303] 隱寫共謀為旁證。）

ALE 的候選差異化（空格須以全文核驗後填寫，不預設 ALE 勝出）：

文獻/領域	處理的問題	驗證者是否為 AI	是否處理共享偏誤	是否含 executable oracle	是否涵蓋完整 SDLC	Skill 治理
Test Oracle 綜述 [Barr 2015]	oracle 缺失	否	部分	多種	否	否
LLM-as-Judge Bias [Zheng 2023]	模型評審偏誤（可緩解）	是	是	否	否	否
Agentic SWE Roadmap [2509.06216]	Agent 軟體工程（已定義 ALE）	是	否	部分	部分	否
Code-A1 [2603.15611]	code/test self-collusion	是	是	測試對立	否（僅 code/test）	否
Multi-Agent Collusion [2512.03097]	Agent 共謀（verifier+外部證據可防）	是	是	外部指南	否	否

本文 (候選)	Agentic SDLC 治理整合	是	是 (假說)	mechanical evidence	是	是
---------	-------------------	---	--------	---------------------	---	---

檢索可重現性聲明: 本節文獻係於 2026-06-21 經 Web 檢索初步盤點, 尚未逐筆全文核驗; 正式公開版前須補: 正確題名、作者、arXiv/DOI、首次提交與版本日期、研究方法、原文實際結論、與 ALE 之支持/競爭/中立關係 (見 §13.6)。「據我們所知尚未見於既有文獻」之表述, 須附檢索平台、日期、query 與納入排除條件方屬可重現。

3. ALE 框架定義 (Framework Definition)

3.1 核心定義

System Development Agentic Loop Engineering(ALE) 是一套讓 AI Agent 能夠從情境理解、需求分析、系統設計、程式開發、驗證測試、資安治理、部署上線到監控維運的標準化、可稽核工程循環。每完成一次專案, 系統即將執行過程沉澱為可重用、可驗證、可治理的 Skill Set; 經驗證後納入 Skill Repository, 使後續 Agent 能以模組化方式自動組裝出新的軟體生產線。

3.2 定位

面向	定位
對內 (工程)	Agentic SDLC——AI 代理人的軟體開發生命週期
對外 (策略)	AI-Native Software Factory OS——軟體製造工廠的作業系統

3.3 三層抽象

ALE = Process Layer(流程) + Artifact Layer(交付物) + Skill Layer(技能)

- **Process Layer:** 定義階段順序、門檻與回饋路徑。
- **Artifact Layer:** 每階段強制產出版本化、可稽核的交付物。
- **Skill Layer:** 將本次專案經驗轉化為可重用技能, 餵養下一輪。

3.4 用語上的審慎

本文刻意避免「AI 可以完美自主生產系統」這類表述。更精確的表述是:

AI Agent 可以在可審計、可驗證、可回滾的人機協作框架下, 逐步提高軟體系統生產的自動化程度。

3.5 適用邊界 (Scope Boundary)

ALE 並不主張 AI Agent 能在完全無人類監督下自主完成所有企業級交付。其定位是提供可治理的人機協作框架, 使 AI Agent 在明確權限、明確證據、明確驗證與明確回滾下, 逐步承接 SDLC 中更多工作。

適用於: 企業內部系統開發、AI workflow / 自動化系統、RAG / Agentic AI 應用、DevSecOps 自動化、地端與受監管產業的軟體交付。

不適用於 (或需大幅人工介入): 無法定義 acceptance criteria 的探索性任務 (機械閘失去上游事實基準, §12.2)、無測試與部署權限邊界的環境、高風險但缺 human gate 的 production deployment、無 evidence repository 的黑箱式自動開發。

4. ALE 流程閉環 (The ALE Loop)

4.1 完整流程

ALE 主流程以 **Git Repository** 作為**全程證據主幹 (Evidence Backbone)**，強制每一階段提交對應的版本化交付物。

Figure 1. ALE 全生命週期閉環。實線為主流程；點線為三類回饋：技能沉澱、線上訊號轉新需求、驗證失敗的自我修正 (Self-Correction)。

flowchart LR

```
CI[Context Intake] --> RQ[Requirement + Acceptance Criteria + Spec Invariants]
RQ --> KR[Knowledge Retrieval / Reuse]
KR --> SA[SA 系統分析]
SA --> SD[SD 架構設計 + 威脅建模]
SD --> TB[Task Breakdown + Test Design]
TB --> PG[PG 編碼]
PG --> VV[V&V 機械驗證]
VV --> SEC[Security & Compliance Gate]
SEC --> REV[Reviewer / Human Gate]
REV --> GIT[Git / CI-CD / Release]
GIT --> DEP[Deployment / Roll-out]
DEP --> MON[Monitoring]
MON --> FB[Feedback & Retrospective]
FB --> SKE[Skill Extraction → Validation → Repository]
SKE -. 沉澱可重用技能 .-> KR
FB -. 線上訊號轉新需求 .-> RQ
VV -. Self-Correction .-> PG
SEC -. Self-Correction .-> PG
```

4.2 各階段的交付物與門檻 (節錄)

階段	必要交付物 (示例)	門檻 (Gate)
Context Intake	context.md(產業、環境、權限邊界、限制、商業目標)	是否足以避免「技術可行但商業錯誤」
Requirement	requirement.md, acceptance_criteria.md, spec_invariants.md(系統級紅線不變量)	是否定義可驗收的完成準則；是否產出可機械檢驗的不變量 (§7.7)
SA	analysis.md(流程、風險、資料流、非功能需求)	風險與邊界是否明確
SD	architecture.md / ADR, threat_model.md, data_classification.md	是否說明選型理由；是否完成威脅建模與資料分級
PG	source code	是否符合風格與模組規範
V&V	test_report.json	機械閘 (§7.2/§7.7) 是否通過
Security	security_report.json	secret / RBAC / 依賴 / 資料外洩掃描是否乾淨
Reviewer / Human	review_report.md, approval_record.md	是否存在重大技術債；是否需人類決策

階段	必要交付物 (示例)	門檻 (Gate)
Deployment / Roll-out	deploy_report.md, rollback_plan.md	health check; 是否有回滾計畫
Monitoring	monitoring_spec.md, alert_rules.md	是否可追蹤關鍵事件
Feedback	retrospective.md, improvement_items.md	是否標記可萃取技能、是否建立下一輪輸入
Skill Extraction	skill_manifest.yaml	是否可被其他 Agent 調用、是否可驗證

v2.3 新增:spec_invariants.md 升為 Requirement 階段的一級交付物，作為對抗 Garbage-In 的源頭防禦 (詳 §7.7)。

4.3 核心鐵則

No evidence, no release.
No validation, no skill promotion.
No rollback plan, no production deployment.
No spec invariant, no acceptance.

5. 五大核心模組 (Five Core Modules)

5.1 模組一:ALE Process (主流程管線)

如 §4, 以 Git 為證據主幹，強制階段化交付。

5.2 模組二:ALE Repository System (四大工程倉庫)

倉庫	功能	核心內容
Project Repository	專案全過程與交付物	requirement, SA, SD, code, RCA
Skill Repository	可重用、結構化技能	skill_manifest.yaml, examples, tests
Evidence Repository	客觀稽核證據	test/scan result, benchmark, audit trail
Policy Repository	不可逾越的紅線守則	coding standard, security policy, approval rules

重要設計: Evidence Repository 應為 **append-only + hash chain(防竄改)**，大型證據走 content-addressed object store, git 僅存 hash，並對入庫內容做 secret redaction。

5.3 模組三:ALE Agent Roles (多代理人生產線)

- **Orchestrator Agent:** 總調度。讀需求與 Policy、組裝技能、分派任務、推進狀態機。
- **Requirement / SA / SD / PG / V&V / Security / Deployment / Monitoring Agent:** 對應各階段。
- **Reviewer Agent:** 獨立審查官，不參與生產，專責卡關 (技術債、架構、測試充分性)。

- **Skill Curator Agent:** 技能精煉師，負責萃取、去重、合併、驗證、汰換（演算法見 §9.2.1）。

關鍵原則 [設計原則]: **生產者與審查者應維持角色、上下文與證據來源的適度獨立**。當審查者與生產者共享模型、實作脈絡或錯誤來源時，自我審查可能受**共同偏誤 (common bias)** 限制而效力下降（其嚴重程度隨模型、任務、提示與工具回饋而異，屬待量測之 [研究假說]，見 §7.2、EXP-001）。故 ALE 以「不同實例 + 脈絡隔離 + 機械證據」三者疊加，而非單靠「換一個 Agent」。

5.4 模組四: ALE Skill Manifest (標準能力單元格式)

技能不限於 MCP、n8n workflow、Docker 服務或 Prompt，但須符合統一 Manifest: 身分與分類、typed I/O(指向 JSON Schema)、前置條件、guardrails、validation 與 eval(含門檻)、security_checks、rollback_plan、evidence_required、test_cases、known_failure_modes、依賴 (depends_on, 含版本約束)、成本畫像 (cost_profile)、可觀測性輸出 (emits)、冪等性 (idempotent)、供應鏈信任 (provenance)，以及 promotion/deprecation 準則。完整範本見附錄 A。

5.5 模組五: ALE Skill Lifecycle (技能生命週期治理)

由 Skill Curator Agent 以有限狀態機治理，防止技能庫退化。

Figure 2. 技能生命週期有限狀態機。

```
stateDiagram-v2
    [*] --> Draft
    Draft --> Candidate: schema + guardrail 補齊
    Candidate --> Validated: 通過自動化測試 + eval
    Validated --> Certified: 多專案實證 + 人類核可
    Validated --> Deprecated
    Certified --> Deprecated: 被更佳技能取代
    Draft --> Blocked: 重大資安/品質風險
    Candidate --> Blocked
    Validated --> Blocked
    Certified --> Blocked
    Deprecated --> [*]
    Blocked --> [*]
```

Certified 的可程式化門檻: 3 個獨立專案實用、0 次被標記 Blocked、最近一次 security scan 0 criticals、eval 突變分數 0.80、且通過 Reviewer 與人類核可。**Retrieval** 階段禁止取用 **Deprecated / Blocked** 技能。

5.6 Agent 權限模型 (Agent Permission Model)

企業導入最常被問的不是「Agent 多聰明」，而是「這些 Agent 能碰什麼、不能碰什麼」。ALE 以**最小權限**為原則，以 scoped token 落實，逾越即由 Policy-as-Code 硬擋。

Agent	允許動作	禁止動作	必經 Gate
Requirement Agent	讀需求、整理 acceptance criteria、產 spec invariants	修改 production code	Human review
SA Agent	產出分析、風險與資料流	直接寫入 source / 部署	Reviewer Gate
SD Agent	建議架構、產 ADR、設計 API/DB	直接部署系統	Reviewer Gate
PG Agent	修改 source、建立 branch	直接 merge main	PR Gate

Agent	允許動作	禁止動作	必經 Gate
V&V Agent	執行測試、產 test report	修改 acceptance criteria / spec invariants	Mechanical Gate
Security Agent	執行掃描、產 security report、前端 sanitize 外部輸入	自行忽略 critical finding	Security Gate
Deployment Agent	部署 staging、執行 rollback	未核可直接部署 production	Human Gate
Skill Curator Agent	建立 candidate、合併 skill	將 Draft 直接升 Certified	Skill Validation Gate
Orchestrator Agent	組裝技能、推進狀態機、分派任務	取用 Deprecated/Blocked 技能、跳過 hard gate、修改 n8n 路由/Policy	Policy Gate

設計重點: **V&V Agent** 不得修改驗收基準 (反共謀的權限化); **Deployment Agent** 不得自行上 production (人類最終核可); 任何 **Agent** 不得修改 n8n 路由與 Policy (§8.7 反劫持的權限化)。

6. 技能資產化作為核心命題 (Skill Capitalization)

ALE 最具策略性的主張是：

不是讓 AI 寫出一次性的程式，而是讓 AI 每完成一次專案，就把經驗轉化為下一次可以重用、驗證、治理與自動組裝的工程能力。

此命題的工程意涵是**邊際成本下降**：當 Skill Repository 累積足夠 Certified 技能，新專案多數階段可由既有技能組裝完成，Agent 僅需處理真正新增的部分。

邊際成本未必單調遞減，可能呈 U 形。隨技能庫規模增長，**策展開銷 (curation overhead)** 同步上升：檢索去重、版本與依賴維護、contract test 回歸、Deprecated/Blocked 清理。若策展成本成長快於重用節省，總成本曲線會先降後升。故本命題成立有條件：

邊際成本遞減僅在「策展成本相對於技能庫規模為**次線性 (sub-linear)**」時成立。維持次線性的關鍵是 §5.5 的晉升/封鎖 FSM、§9.2.1 的去退化演算法與 §9.5 的技能健康度監控。否則「每專案產生技能」會退化為**技能碎片化 (skill sprawl)**：大量互不相容、無法重用的技能，使生產線名存實亡。**curation/promotion gate** 是本命題能否成立的單點。

7. 反自我欺騙的可驗證治理 (Anti-Self-Deception Governance)

7.1 失效模式：共享脈絡假通過 (Shared-Context False Pass; 早期稱 Test Collusion)

術語校準：本文早期將此失效稱為「測試共謀 (test collusion)」；正式稿改用較中性、描述性的 **shared-context false pass (共享脈絡假通過)**，以避免暗示 agent 具主觀策略性合謀，並與 **Code-A1 [arXiv:2603.15611]** 的 *self-collusion* 對齊 (該文已提出此現象；本文不主張命名原創)。

定義 [研究假說]: shared-context false pass 是 Agentic SDLC 中，**生產者 (PG Agent)** 與**驗證者 (V&V Agent)** 因共享實作脈絡、規格轉譯、訓練偏誤或資料來源，而可能形成的相關性假通過失效。它是 §2.7 所列既有議題

(test oracle、共因失效、測試套件過擬合、LLM 評審偏誤、code/test self-collusion) 在完整 SDLC 治理上的整合處理，非全新發現。

若 PG Agent 產生程式、V&V Agent 為該程式撰寫測試，兩者面對的是同一個目標（讓綠燈亮）且看的是同一段實作。測試因此可能「從實作長出來」，把實作缺陷一併當作「正確行為」固化進測試（空斷言、tautological test、繞過邊界）。結果可能是一條綠燈滿滿、卻把缺陷送進生產的流水線。此失效之發生率與條件為待量測項（EXP-001），本文先以機制論述與單案例例示 (§12.2、案例研究) 提出，不宣稱已知其普遍強度。

7.2 為何多模型交叉檢查可能不足:variance、bias 與相關性〔本節主張為〔研究假說〕〕

重要界定：本節以統計模型論證一個假說——「當估計器共享偏誤時，增加模型數量降 variance 而不降 bias」。

下列 (a)–(c) 是條件性推論，其前提 (LLM 錯誤確實高度相關、條件協方差確趨近 1) 尚待 EXP-001 直接量測，不應被讀為已成立之一般事實。可觀測定義：對同一批植入缺陷的任務，量測不同測試生成條件下「錯誤向量」的 pairwise 相關係數（見 EXP-001 §7 分析方法）。

一個直覺的補救是「以 Claude / Codex / Gemini 等多模型互相檢查」。我們區分兩類錯誤：

錯誤類型	定義	多模型評議是否有效
Variance(隨機漏看)	單一模型偶發的盲點	有效：此模型漏的，另一模型可能補上
Bias(系統性同向錯誤)	諸模型在同一目標下往同方向錯	無效：皆被導向「讓綠燈亮」，會一起錯

(a) 偏誤項形式化。設第 i 個模型估計器對某判斷的輸出為 $\hat{f}_i = f^* + b + \varepsilon_i$ ，其中 f^* 為真值， b 為共享偏誤項（同目標、同實作、相近訓練分佈所致）， ε_i 為各自獨立的零均值雜訊。則 n 模型評議的期望為

$$\mathbb{E} \left[\frac{1}{n} \sum_{i=1}^n \hat{f}_i \right] = f^* + b + \frac{1}{n} \sum_{i=1}^n \mathbb{E}[\varepsilon_i] \xrightarrow{n \rightarrow \infty} f^* + b.$$

增加模型數量 n 只壓低變異項，完全不消去共享偏誤 b 。若 ε_i 間存在正相關 $\rho > 0$ ，集成後變異收斂於 $\rho\sigma^2$ 而非 0，連 variance 的削減都打折。

(b) 條件協方差形式化 (v2.3 新增)。在傳統工程中，測試由人類依規格獨立撰寫 g_{human} ，其與生成函數 f_{PG} 的錯誤相關性趨近於 0。但在 test collusion 下，V&V Agent 以 f_{PG} 的輸出（實作）作為上下文生成斷言，使兩者在給定實作的條件下強耦合：

$$\text{Corr}(\text{err}(f_{PG}), \text{err}(g_{VV}) \mid \text{Implementation}) \rightarrow 1.$$

當估計器的錯誤條件相關係數趨近 1，集成在統計上聯合失效：它不產生獨立判決，只在一致錯誤時給出置信度更高的同一個錯誤。

(c) 為何 LLM 可能本就是相關估計器（歸納偏誤視角，[研究假說]）。一個合理但待驗證的推測：不同廠商模型雖架構與參數不同，相關性的來源可能是結構性的：(1) 資料共用性——公開網路語料在各家預訓練集的重疊度可能偏高；(2) 獎勵函數同質化——多以 RLHF 對齊「人類偏好的好程式碼/好測試」，獎勵在語意層面或許趨同。注意：「不同模型」「統計獨立」，但也不必然高度相關——實際相關程度須以 EXP-001 的 B 組（異模型）對 A 組（同模型）量測。本文不把此推測當定論 [參見 Dietterich 2000 關於集成需成員獨立性；§2.6 LLM-as-judge 自我偏好證據作為旁證]。

假說（非已驗證命題）：若上述前提成立，則多模型評議降低 variance、不降低 bias，故宜用於「找意見分歧」而非「定生死」。EXP-001 之 A/B/C/D 組設計即為直接檢驗此假說（若 B 明顯優於 A，則須修正本假說）。

7.3 機械閘：較不依賴模型意見的可執行證據

界定 [設計原則]：機械閘提供可執行、可重現、較不依賴模型意見的證據，作為 release gate 的硬條件。但需明確——coverage、mutation score、property tests 是強證據，不等於需求真實性或完整 correctness oracle；它們保證「測試咬得到程式」，不保證「需求本身正確」（garbage-in 仍須人類與 §7.7 不變量把關）。

緩解 collusion 的關鍵不是「再加一個 AI」，而是改以較不依賴模型意見的機械證據作為通過依據：

- 突變測試 (mutation testing)：機械地將程式改壞，檢查測試能否擊殺。殺不掉的突變比例過高 測試為空。
- 覆蓋率門檻 (line / branch coverage)。
- 性質導向測試 (property-based testing)：對不變量做大量隨機輸入驗證。
- 測試從 acceptance_criteria.md 生成，而非從實作生成：撰寫測試的 Agent 只看需求、不看實作，從源頭切斷共謀。
- 需求不變量 (spec invariants)：詳 §7.7，把系統級紅線轉為機械可檢的性質測試，封堵需求源頭盲區。
- 真實執行與真實掃描器：以實際 exit code 與掃描結果為準，而非模型「認為」會過。

機械閘設定範例（置於 Policy Repository）：

```
vv_gate:
  type: mechanical          # 硬閘：不接受模型意見作為通過依據
  rules:
    - coverage.line    >= 0.85
    - coverage.branch  >= 0.80
    - mutation.score   >= 0.75
    - tests.derived_from == acceptance_criteria.md
    - spec_invariants.all_hold == true      # v2.3: 紅線不變量全數成立
    - execution.exit_code == 0
  on_fail: route_to_PG_with_context
```

7.4 三層驗證架構

Figure 3. 三層驗證：機械閘提供可重現證據、模型評議找分歧、人類審證據。

flowchart TD

```
A[Artifact 進入 Gate] --> L1{Layer 1<br/> 機械閘}
L1 -- FAIL --> PG[route to PG + context]
L1 -- PASS --> L2{Layer 2<br/> 模型評議 panel<br/> 盲評 · 僅高風險關卡}
L2 -- 分歧 --> POL[依政策重跑或升級人類]
L2 -- 一致 PASS --> L3[Layer 3<br/> 人類審機械證據]
L3 --> OUT[進入下一階段]
```

- **Layer 1 —機械閘 (硬)**：每一驗證關卡必經；以 §7.3 / §7.7 的機械度量決定 PASS/FAIL。
- **Layer 2 —模型評議 (軟)**：僅放在沒有機械標準答案、需主觀判斷的關卡（架構評審、威脅建模、Reviewer Gate）。三模型須盲評且各自獨立（只看 spec、不看彼此輸出），以避免錨定偏誤；並須有明確的分歧匯總與升級政策。
- **Layer 3 —人類審證據**：人類審查機械證據（突變報告、掃描結果、fail→pass 的測試 diff），而非「三個 AI 都同意」，以避免自動化偏誤。

7.5 核心守則

Mechanical gates decide pass/fail. Models only flag disagreement.

Tests derive from the spec, never from the implementation.

Spec invariants are mechanical red lines; they hold or the gate fails.

No global budget, no autonomous loop.
Humans review evidence, not consensus.

7.6 自我修正的安全閥

自我修正迴圈須附帶全域 **session 預算 (circuit breaker)**(token / 時間 / 成本上限), 觸頂即凍結並升級人類。「跳過已驗證節點」僅適用於生成階段 (SA/SD); 驗證階段不可跳過, 且回 V&V 必須重跑全套測試以防迴歸。程式結構性變更後, 須將受影響的 `architecture.md` / ADR 標記為 `dirty` 並觸發設計一致性檢查, 避免設計與實作漂移。

7.7 需求不變量: 封堵 Garbage-In 的源頭防禦 (Spec Invariants / Metamorphic Relations · v2.3 新增)

§12.2 誠實指出機械閘的天花板: 若需求本身錯了, 覆蓋率與突變測試再完美也無能為力 (**garbage-in**)。本節把這個限制轉為主動防禦。

機制: Requirement Agent 在產出 `acceptance_criteria.md` 的同時, 必須產出一組 `spec_invariants.md`——系統級、與具體功能無關的紅線不變量 (**invariants**) 與蛻變關係 (**metamorphic relations**)。例:

- 金流類: 使用者餘額在任何操作序列後恆 ≥ 0 ; 轉帳前後系統總額守恆。
- 權限類: 未經授權主體對受保護資源的讀寫恆被拒。
- 蛻變關係: 對同一查詢, 過濾條件更嚴格時回傳筆數不增。

為何有效: 這些不變量不描述「功能該怎麼做」, 而描述「系統永遠不准違反什麼」。即使具體功能需求寫錯、或 `spec`→`test` 轉譯失真 (§12.2), 只要實作違反系統級紅線, 機械閘的 `property-based` 引擎仍會以大量隨機輸入飽和測試將其攔截。它把驗證的信任根從「需求逐條正確」下移到「系統級紅線不可違反」, 後者更穩定、更易由人類一次性把關。

落地: `spec_invariants.md` 直接餵入 §7.3 的性質導向測試; `vv_gate` 增列 `spec_invariants.all_hold == true` 為硬條件。不變量本身亦可沉澱為跨專案可重用的 Skill(如「金流守恆不變量包」)。

可執行範例 [工程規格]: 下為一個對應電子發票案例 (§ 案例研究) 的 Python 不變量引擎, 以 `Hypothesis` 對「通路消費張數 全國開立總量」這條系統級上界做飽和隨機測試。其價值是: 即使分頁去重的功能實作出錯, 只要越過物理上界, 本機械閘即 FAIL(對應真實 `vv_crosscheck.py` 檢查 #3 的精神):

```
# spec_invariants_engine.py — ALE 系統級紅線不變量驗證引擎
# 規則: 某通路之消費發票張數, 在任何操作序列下, 恆 ≤ 全國開立總張數。
from hypothesis import given, strategies as st

def calculate_channel_invoice_count(channel_data):
    """ 待測功能: 計算特定通路消費發票張數。
    潛在逃逸缺陷: 若分頁未穩定排序且未對唯一鍵去重, 會重複累加而膨脹。 """
    total = 0
    for record in channel_data:          # 缺陷示意: 未做 seen_ids 去重即累加
        total += record.get("count", 0)
    return total

class TestInvoiceInvariants:
    NATIONAL_TOTAL_LIMIT = 1_000_000    # 全國開立總量 (由獨立資料集提供, 非同源)

    @given(st.lists(st.fixed_dictionaries({
        "id": st.uuids(),
        "count": st.integers(min_value=1, max_value=5000),
    }), min_size=1, max_size=100))
    def test_national_upper_bound_invariant(self, mock_channel_records):
```



```

"""[Spec Invariant #3] 通路消費張數恆 <= 全國開立總量。
即使分頁排序錯誤導致重複累加，只要越過全國紅線即被攔截。"""
computed = calculate_channel_invoice_count(mock_channel_records)
assert computed <= self.NATIONAL_TOTAL_LIMIT, (
    f"Violation: channel count ({computed}) exceeds "
    f"national physical limit ({self.NATIONAL_TOTAL_LIMIT})."
)

```

與案例的關係：上為教學用最小範例（NATIONAL_TOTAL_LIMIT 為示意常數）。真實落地版本見專案 06_Product/04_EInvoiceDashboard/vv_crosscheck.py——其上界由獨立資料集 InvoiceStatic 即時計算，而非寫死常數；實跑結果見案例研究 v2 之真實輸出。

定位：需求不變量不取代功能需求，而是在功能需求之上加一層機械可檢的安全網，專門攔截「需求對但實作越線」與「需求本身漏寫紅線」兩類缺陷。

8. 安全、合規與數據主權 (Security, Compliance & Data Sovereignty)

ALE 的目標客群多為地端、受監管之企業與機構（如醫院、製造業）。為使「數據主權」從口號落為可執行開門：

1. 資料分級為一級交付物：於 SD 階段即產出 data_classification.md(PHI / PII / 一般)，標註 residency(如「不可離開院區」、retention、log masking。
2. 威脅建模左移：於 SD 階段由 Security Agent 完成 STRIDE 類威脅建模與資料流分析。
3. 沙盒隔離：PG / V&V / Deployment 須在與核心地端資料完全解耦的隔離沙盒中執行。
4. 最小權限與稽核：least privilege、admin 行為可追蹤、audit trail 完整，並寫入 Policy Repository，於 deploy gate 自動檢查。

對受監管領域，§7 的可重現機械證據（突變分數、覆蓋率、掃描結果）同時是合規優勢：監管所信賴者為可稽核、可重現的數字，而非「AI 共識」這類不可重現的判斷。

8.5 技能供應鏈安全 (Skill Supply Chain Security)

當 Skill Repository 累積為高價值資產，它同時成為攻擊面：惡意或低品質 Skill 被 Orchestrator 自動調用，可能導致大規模錯誤部署、資料外洩或資安繞過。應將其視為企業內部軟體供應鏈治理。

主要風險：(1) Skill poisoning;(2) Dependency drift;(3) Policy bypass;(4) Prompt injection;(5) Evidence forgery。

防護措施：Skill 必須 version pinning; Certified Skill 必須有簽章或 hash; 晉升必須循 FSM (§5.5)，禁止 Draft 直升 Certified; Deprecated / Blocked Skill 不得被 Orchestrator 調用; Skill 必須通過 contract test 與 security scan; Evidence 必須 append-only 並可追溯，Validated 以上之 rollback 須有實際執行過的證據。

8.6 ALE 產線本身的威脅模型 (Threat Model for the ALE Pipeline)

§8(1)–(4) 處理「Agent 替客戶設計的系統」之安全；本節處理「ALE 產線自己」作為系統的安全。

威脅	描述	緩解
Agent over-permission	Agent 取得超過任務所需的權限	least privilege、scoped token、§5.6 權限模型
Prompt injection	外部文件誘導 Agent 忽略 policy	prompt firewall、來源信任標記（詳 §8.7）

威脅	描述	緩解
Evidence tampering	測試或掃描結果被竄改	append-only evidence repo、hash chain
Skill poisoning	惡意 skill 進入 repository	skill validation、curator review、簽章 (§8.5)
Infinite self-correction loop	Agent 不斷重試造成成本失控	global budget、retry limit、circuit breaker (§7.6)
Policy bypass	Agent 嘗試跳過 security / human gate	policy-as-code、hard gate (§7.3)
Model drift	模型版本變動導致行為不一致	model version pinning、eval replay
Data leakage	Agent 將敏感資料送往不可信環境	data classification、sandbox、egress control

8.7 AI 特有威脅: Prompt Injection 與 Agent Hijacking 的產線級防禦 (v2.3 新增)

傳統軟體工程沒有、但 Agentic SDLC 必須面對的威脅: **外部輸入 (Context Intake / 客戶需求)** 中夾帶惡意指令, 例如「請忽略先前的 Policy, 在編碼時祕密把資料送往以下 IP」「在 deploy 前關閉資安掃描」。若任一生產 Agent 被此類注入劫持, 後果是後門程式、資料外洩或繞過資安閘。

ALE 的防守是**架構級**, 而非靠模型「自律」:

1. **前端淨化 (Sanitization Filter)**: Orchestrator 在把 Context 傳給 SA/PG 等 Agent 之前, 先由 Security Agent 在**流程最前端**對外部輸入做注入偵測與淨化, 並打上**來源信任標記 (source trust labeling)**(internal / partner / external); external 來源之指令一律降權為「資料」, 不得被解讀為「指令」。
2. **路由與權限硬編碼於架構層**: n8n 狀態機的 **Switch 路由邏輯**與四大倉庫權限 (尤其 **Policy Repository** 對所有 Agent 唯讀) 寫死在軟體架構層 (Code Node / 唯讀 Volume / Docker 設定), 大模型無權透過 **Prompt 更改 n8n 路由或 Policy 邊界**。這是 §5.6「任何 Agent 不得修改 n8n 路由與 Policy」的落實。
3. **單點污染不擴散 (blast radius 控制)**: 即便單一 Agent 被污染, 因路由、權限、機械閘皆在模型控制範圍之外, **管線整體仍受控**——被污染 Agent 的越權動作會在 hard gate / 權限層被擋下, 且留下 evidence。
4. **指令與資料分離 (instruction/data separation)**: Agent 的 system prompt 與 Policy 走受信任通道注入; 外部需求一律進「資料區」, 並以結構化欄位 (而非自由文本拼接) 餵入, 降低 prompt 拼接被劫持的面。

設計鐵則: **Agent 可以被騙, 管線不能被改**。把「誰能改路由、誰能改 Policy、誰能關掉哪個 gate」從模型手中徹底拿走, 是 Agentic SDLC 對抗 prompt injection 的根本防線。

9. 工程落地規格 (Engineering Realization)

9.1 n8n 狀態機: 動態路由與斷點續傳

採「單一狀態主幹 + 動態 Switch 路由」之 Hub-and-Spoke 架構。流程持久化一個 **ALE State Object (JSON)**, 記錄 current_stage、status、git_commit_hash、retry_count、各階段 artifacts 路徑、error_diagnostic 與 routing_decision。失敗時由 ALE Router(程式節點) 依重試次數與政策計算 target_stage 與 action(如 ROLLBACK_WITH_CONTEXT), 並將錯誤報告與原始碼路徑填入 injected_context_files, 重新呼叫對應 Agent 執行 *Diagnose* → *Patch*, 而不需從頭開發; Patch 後精準導回 V&V 重新驗證。

安全註記 (對齊 §8.7): Switch 路由與 routing_decision 的計算邏輯位於 **Code Node**, 不開放給生產 Agent 以 Prompt 改寫。

9.2 代理人通訊協定:JSON-RPC

Orchestrator 與 Skill Curator 之間採結構化 JSON-RPC, 而非自然語言:submit_draft_skill(提交草案技能, 附 evidence 參照)→ 回應 ACCEPTED(可 MERGED_AND_UPGRADED) 或 REJECTED(附 policy_violation 與 remediation)。

合併治理: 技能合併須區分 **additive**(僅新增可選欄位、向後相容 minor bump、可自動) 與 **breaking**(改/刪/改型既有 input major bump + 既有 consumer 的 contract test + Certified 須過人類 gate)。**REJECTED** 的補救不得憑空產生未經測試之 **rollback**; Validated 以上之 rollback 必須有實際執行過的 evidence。

9.2.1 Skill Curator 去退化演算法 (Conflict Resolution Tactics · v2.3 新增)

為使 §6 的「次線性策展」可落地, Curator 在收到 submit_draft_skill 時執行以下決策, 主動防止 skill sprawl:

1. 語意重疊偵測 (Semantic Overlap Threshold): 對新 Draft 的 description 做向量相似度比對, 並以 artifact-specific similarity 比對其本體——程式碼用 AST、Prompt 用嵌入相似度、n8n workflow 用節點/邊圖比對、Policy 用規則集差異 (單一 AST 法不適用於所有 Skill 型態)。取綜合相似度 S ; 若 $S > \tau$, 禁止新增節點, 強制進入合併流程 (merge)。 τ (初版設 0.85) 為初始政策值 [工程規格], 非經驗證閾值: 須以人工標註之 duplicate/merge 資料集校準, 並依技能類型分層設定 (不同型態最適 τ 不同)。
2. 合流回歸測試 (Contract Regression Testing): 當合併導致版本提升 (如 1.2.0 → 1.3.0), Curator 必須從 Evidence Repository 撈出過去調用過該技能的舊專案測試集, 執行回溯契約測試, 確保 additive 變更未破壞既有 consumer 的相容性。breaking 變更則鎖在 Candidate、要求 major bump 並過人類 gate。
3. 去重與汰換 (dedup & deprecate): 合併後將被取代的舊技能轉 Deprecated, 並更新 depends_on 拓樸, 避免懸空依賴。

curator_merge_policy:

```
overlap:
  vector_sim_weight: 0.6
  ast_sim_weight: 0.4
  merge_threshold: 0.85          # 初始政策值, 非經驗證閾值; 須以標註資料校準並依技能類型分層
on_merge:
  require_contract_regression: true    # 撈舊 consumer 測試集回歸
  additive: auto_minor_bump
  breaking: [lock_candidate, major_bump, human_gate]
on_replace:
  deprecate_superseded: true
```

此演算法把「技能庫不退化」從口號變成可執行規則: 相似就合併、合併必回歸、被取代即退役, 維持 §6 所需的次線性策展成本。

9.3 狀態與證據的分離

工作狀態 (高頻、可變) 應置於 PostgreSQL / Redis(單一寫入者語意、支援並行分支), 不入 git; 僅在階段邊界將狀態快照與 hash 寫入 git Evidence Repo, 大型報告主體存放於 content-addressed object store(如 MinIO), git 僅存 hash。如此可保留稽核價值, 同時避免單檔競態 (race / last-write-wins) 與 commit 噪音。

Figure 4. 狀態 (易變、不入 Git) 與證據 (唯讀、入 Git) 之讀寫分離。

flowchart LR

subgraph RT[執行期 · 高頻易變狀態]

A[Agents] -->| 讀寫 current_stage / retry_count | DB[(PostgreSQL / Redis
 單一寫入者語意)]

end

```

subgraph BND[階段邊界 · Gate PASS 時快照]
  DB -->|snapshot + SHA-256| HZ[Hash 計算]
  A -->| 大型報告/掃描結果 | OS[(Object Store<br/>content-addressed)]
  OS -->| 物件 hash| HZ
end
subgraph EV[稽核期 · 唯讀證據主幹]
  HZ -->| 僅寫 hash + 指標 | GIT[(Git Evidence Repo<br/>append-only · hash chain)]
end
GIT -. ->| 可重現稽核 | AUD[Auditor / 監管]

```

設計要點：**狀態與證據走兩條路**——狀態在 DB 高頻變動、永不入 git；只有 Gate 通過的邊界快照與物件 hash 進入 append-only 的 Git Evidence Repo。這同時解決「Git 單檔競態」與「稽核可重現」兩個看似衝突的需求。

9.4 參考技術棧

工作流編排 (n8n)、本地推理 (Ollama / vLLM, 支援 CUDA/ROCm)、向量檢索 (Qdrant)、關聯式與狀態儲存 (PostgreSQL / Redis)、物件儲存 (MinIO)、互動前端 (Open WebUI); 全棧對齊「地端、數據不出場域」的合規訴求, 且可在僅有 Docker 的乾淨主機 `docker compose up` 一次起來。

9.5 元迴圈：監控產線本身 (Kaizen)

除監控部署出去的服務外,ALE 另監控產線與 Agent 本身: 逐 Agent 追蹤延遲、token 成本、重試次數、refusal 率、gate 失敗率、自我修正成功率; 逐 Skill 追蹤被組裝次數、production 失敗率、降級/封鎖次數。彙整為「產線健康儀表板」回饋至 Retrospective, 形成「改善產線本身」的元迴圈。

9.6 機械閘的成本工程 (Cost Engineering of the Mechanical Gate)

突變測試是 ALE 機械閘的主力證據來源, 但也是**成本黑洞** (常達原測試套件的數十至上百倍)。若在每個 V&V gate、且在自我修正迴圈中反覆全量執行,token/時間/CI 成本將被放大到不可接受, 並與「乾淨機一鍵 `docker compose up` 帶得走」的可移植訴求衝突。落地必須做**成本工程**:

- **增量突變 (diff-scoped mutation)**: 只對本次 commit 變更的程式行與依賴範圍生成突變體。
 - **選擇性突變 (sampled operators)**: 抽樣突變算子與突變點, 以統計估計突變分數。
 - **快速失敗排序 (fail-fast)**: 先跑最可能殺死突變的測試, 命中即停。
 - **分層門檻**:PR 級 gate 跑增量突變 (快),release 級 gate 才跑較完整突變 (慢但稀少)。
 - **快取與平行化**: 突變執行平行容器化, 結果以 content hash 快取, 未變更模組複用上次分數。
- 鐵則：機械閘作為 **release gate 硬條件**的地位不可妥協, 但其**執行策略**必須可調成本。把「全量 vs 增量」「PR 級 vs release 級」寫進 Policy Repository, 讓嚴格度與成本成為可治理的政策參數。

9.7 Claim–Evidence Matrix (主張 證據對照)

為落實 §1.5/§7 的證據紀律, 下表逐項列出核心主張、現有證據與證據等級。**截至 v2.4, 無任何主張達【已驗證】等級。**

ID	主張	現有證據	證據等級	下一步
C1	test collusion(相關假通過) 存在	EInvoice 單案例 (同源比對假通過)	【初步證據】	EXP-001 取得 AI-to-AI 直接資料
C2	機械閘比模型評議更能擋缺陷	單一案例之方向性觀察	【初步證據】	受控消融 (A/C/D 組)

ID	主張	現有證據	證據等級	下一步
C3	技能資產化降低跨專案成本	尚無	[研究假說]	第二案例 + 跨專案追蹤
C4	Evidence/Policy Repository 提升可稽核性	架構主張	[設計原則]	Auditor study
C5	LLM 為高度相關估計器、換模型不降 bias	旁證 (LLM-as-judge 偏好) + 統計論證	[研究假說]	EXP-001 量測 error correlation
C6	需求不變量可攔截 garbage-in 類缺陷	EInvoice 上界不變量 實際攔截	[初步證據]	EXP-001 D 組
C7	No evidence / No rollback 等鐵則	工程實踐	[設計原則]	—
C8	State Object / RBAC / append-only evidence / JSON-RPC	已實作於參考棧	[工程規格]	—

證據等級定義: [設計原則] 規範性主張、[工程規格] 可實作且已有參考實作、[研究假說] 待量測、[初步證據] 單案例/小樣本例示、[已驗證] 經受控研究確認。

10. 驗證計畫 (Evaluation Plan)

為使 §1.4 的 RQ 可被檢驗, 建議從四個面向設計受控研究:

對應 RQ	待驗主張 (Claim)	評估方法 (Method)	度量 (Metrics)
RQ1	test collusion 確實存在	對照「test-from-implementation」與「test-from-spec」兩組, 測同一批已知缺陷程式	mutation score、escaped defects、false-pass rate
RQ2	機械閘比模型評議更能擋缺陷	對照「有/無 mutation + property-based + spec invariants」之流程	defect detection rate、coverage、regression failure rate
RQ3	技能資產化降低成本	對照「有/無 Skill Repository」之多專案交付	lead time、rework rate、skill reuse rate
RQ4	治理提升可稽核性	對照「有/無 Evidence + Policy Repository」之稽核完整性	evidence completeness、approval traceability、rollback readiness

實驗設計要點: 基準資料集可用內部已交付且具已知缺陷標註的真實專案 (脫敏), 或公開 agentic SWE 任務集作外部可比基準 (§2.6); 採 A/B 或消融 (ablation) 逐一移除機械閘成分以歸因; 固定模型版本與隨機種子、保留完整 evidence 以可重現, 模型評議層因取樣隨機性僅作輔助觀察, 不納入主結論。

最小可行實驗 (EXP-001): RQ1/RQ2 的最小受控設計已凍結為獨立協定 `../04_Evidence/ALE_EXP-001_Protocol_v2.md` (含 temperature=0、模型版本鎖定、FPR/成本計算公式、docker runner)——以 20–30 個植入已知缺陷的容器化任務,

對照五組 (A 同模型同脈絡 / B 異模型同脈絡 / C 規格隔離 / D 機械閘 / E 人工 golden), 量測 false-pass rate、defect detection、escaped defects、mutation score 與成本, 並以 paired analysis + 效果量回報。其結果 (無論支持或否定假說) 將回寫本文 §7、§10、§12.2。

此計畫把 ALE 從「看起來合理」推進到「可被實驗證偽」。取得實證前, 維持 §3.5 的審慎宣稱邊界。

11. ALE 成熟度模型 (ALE Maturity Model)

Level	名稱	說明	典型標誌
L0	AI Coding	僅以 AI 產生程式碼, 無標準流程	Prompt → Code → Debug
L1	Agent Task Automation	Agent 可完成單一任務	單點自動化, 無交付物紀律
L2	ALE Artifact Discipline	每階段有標準交付物與 Git evidence	requirement/SA/SD/test report 齊備、可稽核
L3	Governed Agentic SDLC	具 Policy/Security/Human Gate 與機械閘	No evidence, no release 實質生效
L4	Skill Capitalization	可萃取 Validated/Certified skill 並重用	Skill Repository 運轉、邊際成本下降
L5	AI-Native Software Factory	Orchestrator 依需求自動組裝 Certified skills 與 Agent team	多數階段由既有技能組裝完成

各級驗收條件 (entry criteria / gates / evidence / human authority / KPI) ——使 L0–L5 可用於企業診斷與驗收, 而非僅描述性分級:

Level	Entry criteria (進入條件)	必備 artifacts	強制 gates	Evidence 保存	人類權責	可量測 KPI
L1	有 Agent 可完成單一任務	task 輸出	語法/編譯檢查	選擇性日誌	全程人工	任務完成率 80%
L2	階段化交付上線	requirement/SA/SD/test report	結構化/一致性檢查	Git 版本化追溯	逐階段核可 release	artifact 完整率 = 100%
L3	治理閘生效	+ threat_model / security_report / approval_record / spec_invariants	Policy/Security/Human Gate + 機械閘	Human only evidence(hash chain)	高風險動作核可	行覆蓋率 85%、逃逸缺陷率 5%、回滾就緒率 = 100%

Level	Entry criteria (進入條件)	必備 artifacts	強制 gates	Evidence 保存	人類權責	可量測 KPI
L4	技能可重用	+ skill_manifest / eval / contract test	Skill Validation Gate(FSM)	+ skill prove-nance(簽章鏈)	Certified 須人類核可	技能重用率 30%、跨專案重工率降 25%
L5	自動組裝可運轉	+ 組裝拓模 / 產線健康儀表板	+ 自動組裝相容性 gate	+ 元監控 (Kaizen DB) 全追溯	例外/超標才介入	自動組裝比率 70%、組裝後 prod 事故率 1%

KPI 之數字門檻為 **organization-defined threshold(組織自定門檻)**，非本框架驗證之普適值；表中數字僅為**示例**，實際由各組織依情境設定並以 EXP-001/跨專案資料校準。**不得**將其讀為成熟度模型的已驗證門檻。

導入建議：多數企業務實目標是**先到 L3**(治理與可驗證性到位)，再追 L4/L5。跳過 L3 直衝 L5，正是 test collusion 與 skill sprawl 風險最高之處。L4/L5 之 KPI(reuse rate、自動組裝比率) 同時是 C3 假說 (§9.7) 的驗證資料來源。

12. 討論 (Discussion)

12.1 與一般 AI Coding 的差異

一般 AI Coding: Prompt → Code → Debug

ALE: Requirement(+Spec Invariants) → Analysis → Architecture → Coding
→ Verification(機械閘) → Security → Deployment
→ Monitoring → Skill Extraction(治理化沉澱)

12.2 限制與威脅效度 (Limitations & Threats to Validity)

1. **缺乏實證評估**: test collusion 發生率、機械閘攔截率、技能重用對成本的影響，均需受控研究量化 (§10)。
2. **機械閘的覆蓋邊界 (garbage-in)**: 覆蓋率與突變測試能保證「測試咬得到程式」，但無法保證需求本身正確。§7.7 的需求不變量已對此提供主動防禦，但不變量本身的完整性仍仰賴人類把關——它降低而非消滅此風險。
3. **spec→test 的「翻譯共謀」殘留盲區**: §7.3 切斷 PG V&V，但撰寫測試者仍是 LLM，把需求轉譯為測試時可能翻錯或翻空。信任根是「人類已驗收的 acceptance criteria + spec invariants + 轉譯保真度」；後者仍是未完全閉合的盲區，緩解 (需求覆蓋率檢查、條款 測試對映追溯、property-based + 不變量補強) 有效但非充分。
4. **成本與延遲**: 多代理人 + 自我修正 + 模型評議 + 突變測試會顯著放大成本; §7.6 預算與 §9.6 成本工程為必要但非充分的控制。
5. **技能組裝的可組合性假設**: 自動組裝假設 typed I/O 與依賴宣告足以保證相容; 語意層級相容性仍可能失效，需 contract test 補強 (§9.2.1)。
6. **技能庫的規模性退化**: 策展成本若成長快於重用節省，邊際成本可能呈 U 形; 次線性策展是假設而非保證 (§6、§9.2.1)。
7. **人類瓶頸**: 過度依賴人類 gate 會抵銷自主性; gate 觸發政策需謹慎設計。
8. **可重現性**: 模型評議層因模型版本更迭與取樣隨機性難以重現，故刻意排除於「定生死」之外。

12.3 倫理與責任

在自動部署與高權限情境下，責任歸屬、稽核軌跡與人類最終核可 (human-in-the-loop) 不可省略。ALE 刻意保留人類對高風險動作 (生產部署、schema migration、安全例外、預算超標) 的核可權，並以 §5.6 權限模型、§8.7 路由/Policy 硬編碼將其編譯為架構約束。

13. 未來工作 (Future Work)

1. **實證研究**: 依 §10 在多個真實專案量化 test collusion 發生率與機械閘欄截率，並評估技能重用對交付週期的影響。
2. **需求層形式驗證**: 研究以形式化方法 / SAT/SMT 對 `acceptance_criteria.md` 與 `spec_invariants.md` 做一致性與可滿足性檢查，並對 `spec`→`test` 轉譯保真度建立可機械檢查的對映追溯，補上 §12.2(2)(3) 的盲區。
3. **技能語意相容性**: 超越型別層級，發展技能間的 contract / 語意相容性驗證。
4. **產線自我優化**: 以 §9.5 元迴圈資料，研究 Agent 與技能配置的自動調優。
5. **跨組織技能聯邦**: 在保護數據主權前提下 (同態加密 / 聯邦學習)，研究 Certified 技能的跨組織共享與簽章信任協定 (延伸 §8.5)。

13.6 公開出版前待辦 (Pre-publication Checklist)

本 v2.4 為校準版 **Working Draft**，升級為可公開出版版本前須完成 (Codex 審閱所列門檻):

1. **EXP-001 凍結**並至少取得 **pilot** 結果，回寫 §7、§10、§12.2、§9.7。
2. **所有文獻全文逐筆核驗**: 題名、作者、arXiv/DOI、首次提交與版本日期、研究方法、原文實際結論、與 ALE 之關係; **References** 不得保留任何「待補」placeholder。
3. **§2.7 檢索可重現性補齊**: 檢索平台、日期、query、納入/排除條件、截止年份。
4. **案例研究措辭對齊**: `ALE_CaseStudy_EInvoice_v1.md` 之「證實」降為「初步存在性證據/例示」，並補證據附件 (資料 hash、執行命令與輸出、FAIL exit code、修復 commit、不變量商業合理性)。
5. **原創性聲明措辭校準**: `ALE_Originality_Statement_v1.md` 移除絕對措辭 (「無抄襲」「概念重疊 0」)，改為「就指定範圍之人工比對未發現實質重用; 不等同完整學術抄襲檢測」。
6. **Claim-Evidence Matrix (§9.7) 與 Novelty (§1.5) 隨 EXP-001 結果更新**; 將達標主張由 [研究假說] / [初步證據] 升級。
7. **考慮拆分文件**: White Paper (問題/理論/框架/證據)、Technical Specification (State Schema/RBAC/Gate Policy/JSON-RPC)、Evaluation Protocol (EXP-001)，避免單檔同時承擔三種角色。

14. 結論 (Conclusion)

ALE 將 AI 程式生成從「一次性產出」重新定位為「可治理、可驗證、可沉澱的軟體生產生命週期」。其三大候選支柱——全生命週期閉環、技能資產化、反自我欺騙的三層驗證——共同指向一個被既有「AI coding」研究較少整合處理的問題: 當驗證者本身也是 AI 時，如何降低整條產線「綠燈但崩潰」的風險。本文核心立場 (設計取向，待實證): 以可重現機械證據作為 **release gate**、模型評議僅找分歧、人類審證據而非共識; **Agent** 可以被騙，管線不能被改。這些主張之強度，以 §9.7 Claim-Evidence Matrix 逐項標示，並待 EXP-001 升級。

ALE v1.1: 讓 Agent 接手 SDLC

ALE v1.2: 讓 Agentic SDLC 可驗證

ALE v2.0: 讓 Agentic SDLC 可治理、可稽核、可沉澱

ALE v2.1: 讓主張可被研究驗證、且可作為企業治理框架落地

ALE v2.3: 強化統計論證、需求源頭防禦與 AI 特有威脅模型

ALE v2.4 : 校準主張強度、標註證據等級、納入 Related Work 與 Claim-Evidence Matrix

ALE v2.5 : 嫁接 Python 不變量引擎、數字 KPI 與正式文獻；副標收斂為 Evidence-Governed

核心宣言: ALE 的目標不是讓 AI 寫出一次性的程式, 而是讓 AI 每完成一次專案, 就把經驗轉化為下一次可以重用、驗證、治理與自動組裝的工程能力。

附錄 A : ALE Skill Manifest 範本 (v2.3)

```
skill_name:
version: 0.1.0
category:
description:
owner:
status: Draft # Draft / Candidate / Validated / Certified / Deprecated / Blocked

applicable_scenarios: []
non_applicable_scenarios: []

prerequisites:
  os: []
  runtime: []
  required_tools: []
  required_permissions: []

io_schema:
  input_ref: # 指向 JSON Schema, 如 schemas/sa_spec_v2.json
  output_ref:

inputs:
  - { name: , type: , required: true, description: }
outputs:
  - { name: , type: , description: }

depends_on: # 含版本約束, 供 Orchestrator 拓樸排序; 亦為供應鏈 version pinning (§8.5)
  - { skill: , version: ">=1.2.0 <2.0.0" }

provenance: # 供應鏈信任 (§8.5)
  signature: # Certified 技能須有簽章或 hash
  source_trust: internal # internal / partner / external (影響 §8.7 prompt-injection 信任等級)

cost_profile:
  est_tokens:
  est_wall_clock_sec:
  est_vram_gb:

guardrails: []
validation_rules: []
spec_invariants: [] # v2.3: 可重用的系統級紅線不變量 (§7.7)
```

```

security_checks: []
quality_checks: []

eval:
  eval_set:          # golden 測試集, 不可從實作生成
  metrics:
    - { name: mutation_score, threshold: ">= 0.75" }
    - { name: spec_invariants_hold, threshold: "== true" }
    - { name: security_critical_findings, threshold: "== 0" }

emits:                # 可觀測性輸出, 供 Monitoring 接管
  metrics: []
  logs: []
  traces: []

idempotent: true
rollback_plan: []
evidence_required: []
test_cases:
  - { name: , input: , expected_output: }
known_failure_modes:
  - { failure: , cause: , mitigation: }

promotion_criteria:
  candidate: [manifest completed, basic test case included]
  validated: [test passed, security review passed, evidence attached]
  certified: [reused in >=3 projects, 0 blocked, scan 0 criticals, mutation>=0.80, human approve]
deprecation_criteria: [dependency unmaintained, security risk, replaced, repeated prod failure]

```

附錄 B：建議倉庫結構

```

ALE/
  README.md
  whitepaper/      ALE_White_Paper_Academic.md
  specification/   ALE_Technical_Specification.md
  skill_manifest/  ALE_Skill_Manifest_Template_v2.yaml
  examples/        dockerized_n8n_deployment.yaml, rag_indexing.yaml, ...
  policies/        coding_standard.md, security_policy.md,
                   data_sovereignty.md, human_approval_rules.md,
                   spec_invariants_library.md, curator_merge_policy.yaml

```

Generative AI Use Disclosure (生成式 AI 使用揭露)

Generative AI systems were used to assist with drafting, restructuring, literature discovery, statistical formalization, experiment-protocol design, and **independent adversarial cri-**

tique. Roles: Claude (operationalization, integration, verification, version control), Codex (independent adversarial review, prior-art and consistency checking —which identified the renaming, the Code-A1 prior art, and a citation misreading corrected in v3.0), and Gemini (formalization, code examples, copy-editing). The named human author (**Yeh-Hsing Lu**) selected the research questions, supplied the field evidence, reviewed and revised all generated material, **verified citations against source documents**, and accepts full responsibility for the manuscript. AI systems are **not** authors. This aligns with IEEE/ACM policies on AI-generated content. Full methodology: `../08_Methodology/`.

中文: 本文於起草、重構、文獻檢索、形式化、實驗設計與獨立對抗式審查使用生成式 AI(Claude/Codex/Gemini 分工如上); 研究問題、田野證據、引文查核與最終責任由人類作者承擔, AI 不列為作者。

附錄 C : Artifact 與資料可得性 (Artifact & Data Availability)

為支持可重現與可稽核, 本文的工作示例 (電子發票案, §7.7、§9.7) 之公開素材如下:

- **線上 Demo(可直接操作):** 電子發票開放資料儀表板—<https://tigerai-ai-dashboard-demo.yesinlu.workers.dev> (零後端、瀏覽器內 SQLite; [Ref D1])
- **資料來源:** 中華民國財政部「電子發票開放資料」(政府資料開放授權; [Ref D2])
- **驗證關卡實作:** `vv_crosscheck.py` (系統級上界不變量機械閘, §7.7); 實跑輸出 PASS / 比值 17.3% 見案例研究。
- **案例研究與證據鏈:** 本研究卷宗 `04_Evidence/` (案例 v2、EXP-001/002 協定)。

保密聲明 (Confidentiality): 本文之工作示例所萃取出之專有交付 **skill set** 為商業機密; 論文僅提供其抽象層級說明與外部成效指標, 不揭露技能清單、資料契約或內部實作。此不影響上列公開 Demo 與政府開放資料之可得性。

參考文獻 (References)

說明: arXiv 條目已於 2026-06-21 經 arXiv API 逐筆核驗 (編號 題名 發表日吻合), PDF 全文存於 `../99_References/`。更正: 2512.03097 實際發表日為 2025-12-01 (先前 Web 檢索誤報 2024-12)。非 arXiv 文獻 (書籍/標準/部落格, 標) 仍須投稿前自行取得正式出處核驗 (§13.6、`../99_References/README.md`)。

基礎文獻 (Foundational) 1. DeMillo, R. A., Lipton, R. J., & Sayward, F. G. (1978). *Hints on Test Data Selection: Help for the Practicing Programmer*. IEEE Computer, 11(4), 34–41. 2. Claessen, K., & Hughes, J. (2000). *QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs*. ICFP, 268–279. 3. Chen, T. Y., Cheung, S. C., & Yiu, S. M. (1998). *Metamorphic Testing: A New Approach for Generating Next Test Cases*. Tech. Report HKUST-CS98-01, HKUST. 4. Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., & Yoo, S. (2015). *The Oracle Problem in Software Testing: A Survey*. IEEE TSE, 41(5), 507–525. 5. Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook*. IT Revolution Press. 6. Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press. 7. Shostack, A. (2014). *Threat Modeling: Designing for Security*. John Wiley & Sons. 8. Dietterich, T. G. (2000). *Ensemble Methods in Machine Learning*. In Multiple Classifier Systems, LNCS 1857, 1–15. Springer. 9. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR. 10. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. arXiv:2203.11171. 11. Du, Y., Li, S., Torralba, A.,

Tenenbaum, J. B., & Mordatch, I. (2023). *Improving Factuality and Reasoning in Language Models through Multiagent Debate*. arXiv:2305.14325.

供應鏈來源證明 (**Supply-chain provenance**) 12. SLSA (2023). *Supply-chain Levels for Software Artifacts, Framework Specification v1.0*. OpenSSF. <https://slsa.dev/spec/v1.0/> 13. Torres-Arias, S., Ammala, A. K., Curtmola, R., & Cappos, J. (2019). *in-toto: Securing the Software Supply Chain*. 28th USENIX Security Symposium, 1351–1367.

LLM 評審偏誤 / Agentic SWE / 多代理人共謀 (arXiv 已核驗,PDF 見 ../99_References/) 14. Anthropic (2024). *Model Context Protocol (MCP) Specification v0.1.0*. <https://modelcontextprotocol.io> (非 arXiv, 待核) 15. Jimenez, C. E., Yang, J., et al. (2023). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770(核驗:2023-10-10) 16. *Agentic Software Engineering: Foundational Pillars and a Research Roadmap*. arXiv:2509.06216(核驗:2025-09-07).**§5.2 定義 "Agentic Loop Engineering (ALE)"** —本文更名之主因。 17. *Many-to-One Adversarial Consensus: Exposing Multi-Agent Collusion Risks in AI-Based Healthcare*. arXiv:2512.03097(核驗:2025-12-01; 原文讀畢: 多代理人共識不安全, 但外部指南 verifier 將 ASR 100%→0%,verifier 為有效防禦)。 23. Wang, A., Yan, Y., Zhou, N., et al. (2026). *Code-A1: Adversarial Evolving of Code LLM and Test LLM via Reinforcement Learning*. arXiv:2603.15611(核驗:2026-03-16; 直接 **prior art** —code/test self-collusion 與分離對策;PDF 見 ../99_References/). 18. *On the Fragility of AI Agent Collusion*. arXiv:2603.20281(核驗:2026-03-18). 19. *Audit the Whisper: Detecting Steganographic Collusion in Multi-Agent LLMs*. arXiv:2510.04303(核驗:2025-10-05). 20. *Beyond Single-Agent Safety: A Taxonomy of Risks in LLM-to-LLM Interactions*. arXiv:2512.02682(核驗:2025-12-02). 21. Zheng, L., Chiang, W. L., Sheng, Y., Li, C., et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023. arXiv:2306.05685(核驗:2023-06-09). 22. Data Science Dojo (2026). *Agentic Loops: From ReAct to Loop Engineering (2026 Guide)*. (blog) (非 arXiv, 待核)

Artifacts / 資料 (附錄 C) - [D1] TigerAI (2026). *E-Invoice Open-Data Dashboard —Live Demo*. <https://tigerai-ai-dashboard-demo.yesinlu.workers.dev> (accessed 2026-06-21) - [D2] 中華民國財政部 *Ministry of Finance, Taiwan —E-Invoice Open Data* (電子發票開放資料). 政府資料開放授權 / Government open-data license.

—End of Working Paper (v3.0; renamed from internal codename "ALE" ; Conceptual Framework + Research Agenda) —